强化学习：从MDP到Q-Learning

#强化学习 #MDP #Q-learning

一、强化学习的核心思想

强化学习（Reinforcement Learning，RL）的目标是让智能体（Agent）在与环境（Environment）的交互中，通过不断试错来学习最优策略（Policy），从而最大化长期累积奖励。

- 智能体：学习和决策的主体。
- 环境：智能体所处的外部世界，会根据智能体的动作发生变化。
- 策略 π：智能体的行为函数，即“在状态 s 下，选择动作 a 的概率” $\pi(a|s)$。
- 目标：最大化未来奖励的期望总和。

二、马尔可夫决策过程（MDP）：RL的数学框架

MDP 是对强化学习问题的标准数学抽象，它假设环境的下一个状态只依赖于当前状态和动作，而与更早的历史无关（马尔可夫性）。

1. MDP 五元组 (S, A, P, R, γ)

符号	名称	含义
S	状态空间	所有可能状态的集合。
A	动作空间	所有可选动作的集合。
P	状态转移函数	$P(s' \mid s, a)$，在状态 s 执行动作 a 后，转移到状态 s' 的概率。
R	奖励函数	$R(s, a, s')$，在状态 s 执行动作 a 并转移到 s' 后获得的即时奖励。
γ	折扣因子	$0 \leq \gamma \leq 1$，用于权衡眼前奖励和未来奖励的重要性。

2. 价值函数（Value Function）

价值函数衡量一个状态（或状态-动作对）的“好坏”程度，即从该状态开始，遵循某个策略所能获得的期望累积奖励。

- 状态价值函数 $V^\pi(s)$：

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right]$$

表示在状态 s 下，遵循策略 π 所能获得的期望长期回报。

- 动作价值函数 $Q^\pi(s,a)$：

$$Q^\pi(s,a) = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty}\gamma^k R_{t+k+1} \mid S_t = s, A_t = a\right]$$

表示在状态 s 下执行动作 a，之后遵循策略 π 所能获得的期望长期回报。

三、贝尔曼方程：价值的递推关系

贝尔曼方程是价值函数的核心递推式，它将当前价值与未来价值联系起来，是所有强化学习算法的理论基础。

1. 贝尔曼期望方程（评估当前策略）

用于计算在给定策略 π 下，每个状态的价值。

- 对于状态价值函数：

$$V^\pi(s) = \sum_a \pi(a|s)\sum_{s'} P(s'|s,a)\left[R(s,a,s') + \gamma V^\pi(s')\right]$$

- 对于动作价值函数：

$$Q^\pi(s,a) = \sum_{s'} P(s'|s,a)\left[R(s,a,s') + \gamma\sum_{a'}\pi(a'|s')Q^\pi(s',a')\right]$$

2. 贝尔曼最优方程（寻找最优策略）

最优价值函数 $V^*(s)$ 和 $Q^*(s,a)$ 是在所有可能策略中能获得的最大价值。

- 最优状态价值函数：

$$V^*(s) = \max_a \sum_{s'} P(s'|s,a)\left[R(s,a,s') + \gamma V^*(s')\right]$$

- 最优动作价值函数：

$$Q^*(s,a) = \sum_{s'} P(s'|s,a)\left[R(s,a,s') + \gamma\max_{a'} Q^*(s',a')\right]$$

> 关键点：max 算子意味着，在未来的每一步，我们都会选择能带来最大价值的动作，这定义了最优策略。

四、Q-Learning：一种经典的无模型算法

当环境的转移概率 P 未知时，我们无法直接使用动态规划（DP）方法，这时就需要无模型（Model-free）算法，如 Q-Learning。

1. 核心思想

Q-Learning 的目标是直接学习最优动作价值函数 $Q^*(s,a)$，而无需先学习环境模型。它通过与环境交互，采样得到 (s,a,r,s') 四元组，然后用这些样本来更新 Q 表。

2. Q-Learning 更新公式

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[R + \gamma \max_{a'} Q(s',a') - Q(s,a) \right]$$

- 学习率 α：控制每次更新的步长，α 越大，新信息覆盖旧信息的速度越快。
- 目标值：由即时奖励 R 和对未来最优价值的估计 $\gamma \max_{a'} Q(s',a')$ 组成。
- 更新过程：用时序差分误差（TD Error）来修正旧估计。

3. 算法流程

1. 初始化：创建一个 Q 表，将所有 $Q(s,a)$ 的值初始化为 0 或小的随机数。
2. 循环 episodes：
   3. 初始化状态 s。
   4. 循环 steps：
      1. 使用 ϵ-贪心策略选择动作 a。
      2. 执行动作 a，得到奖励 r 和下一个状态 s'。
      3. 根据公式更新 Q 表。
      4. 更新当前状态 $s \leftarrow s'$。
      5. 如果 s 是终止状态，则结束该 episode。

五、Q-Learning 代码示例（Python）

```python
import gymnasium as gym
import numpy as np

# 1. 创建环境（注意使用新的库名 gymnasium）
env = gym.make("CliffWalking-v0")

# 超参数
alpha = 0.1    # 学习率
gamma = 0.99   # 折扣因子
epsilon = 0.1  # 探索率
episodes = 500

# 初始化 Q 表：[状态数，动作数]
# CliffWalking 有 48 个状态（4x12 栅格），4 个动作
Q = np.zeros([env.observation_space.n, env.action_space.n])
```

```python
for episode in range(episodes):
    # 最新 API: reset 返回 (state, info)
    state, info = env.reset()
    done = False

    while not done:
        # epsilon-greedy 策略选择动作
        if np.random.uniform(0, 1) < epsilon:
            action = env.action_space.sample()  # 探索
        else:
            action = np.argmax(Q[state])        # 利用

        # 最新 API: step 返回 5 个值
        # terminated: 掉进悬崖或到达终点
        # truncated: 步数达到上限 (默认通常是 100 步)
        next_state, reward, terminated, truncated, info = env.step(action)

        # 只要其中一个为 True, 本轮结束
        done = terminated or truncated

        # 核心更新公式 (TD Error / 贝尔曼方程)
        # Q(s,a) = Q(s,a) + alpha * [R + gamma * max(Q(s')) - Q(s,a)]
        best_next_action = np.max(Q[next_state])
        td_target = reward + gamma * best_next_action
        Q[state, action] += alpha * (td_target - Q[state, action])

        state = next_state

    if (episode + 1) % 50 == 0:
        print(f"Episode {episode + 1} 训练中...")

print("训练完成! ")
```